PSI- 3451: Respostas e Instruções para Entregáveis - Aula 3

Nome do Aluno: Felipe Uehara Gondo **No. USP: 13680612**

Observação: <u>associe o(s) nome(s) dos arquivos subidos à enumeração as seções (itens) da apostila, replicados neste questionário</u>.

Observação: todas as curvas devem estar com boa resolução e os nomes de sinais visíveis e identificáveis.

Para os itens 1) a 5), o(a) aluno(a) deve apresenta as mesmas transições definidas na sua atividade da aula 2, casos de distância de Hamming 1, dadas como A-→ B-→ C ou 000-→ 001-→ 011

O(A) aluno(a) poderá simular também outras transições que achar adequadas para os seus testes.

item 1) full_adder_3

- **Fazer upload de uma ou mais curvas com o correto funcionamento do módulo. Anote os tempos de atraso entre entradas e nós intermediários, e entre estes e as saídas.**
- **No quadro abaixo, discuta se a descrição VHDL gera as mesmas saídas dos full_adder_1 e full_addder_2 da aula 2. Discuta eventuais diferenças nos tempos de atraso e justifique pelo código VHDL.**

Ao simular a sequência de soma Hamming 1, os resultados foram os seguintes:

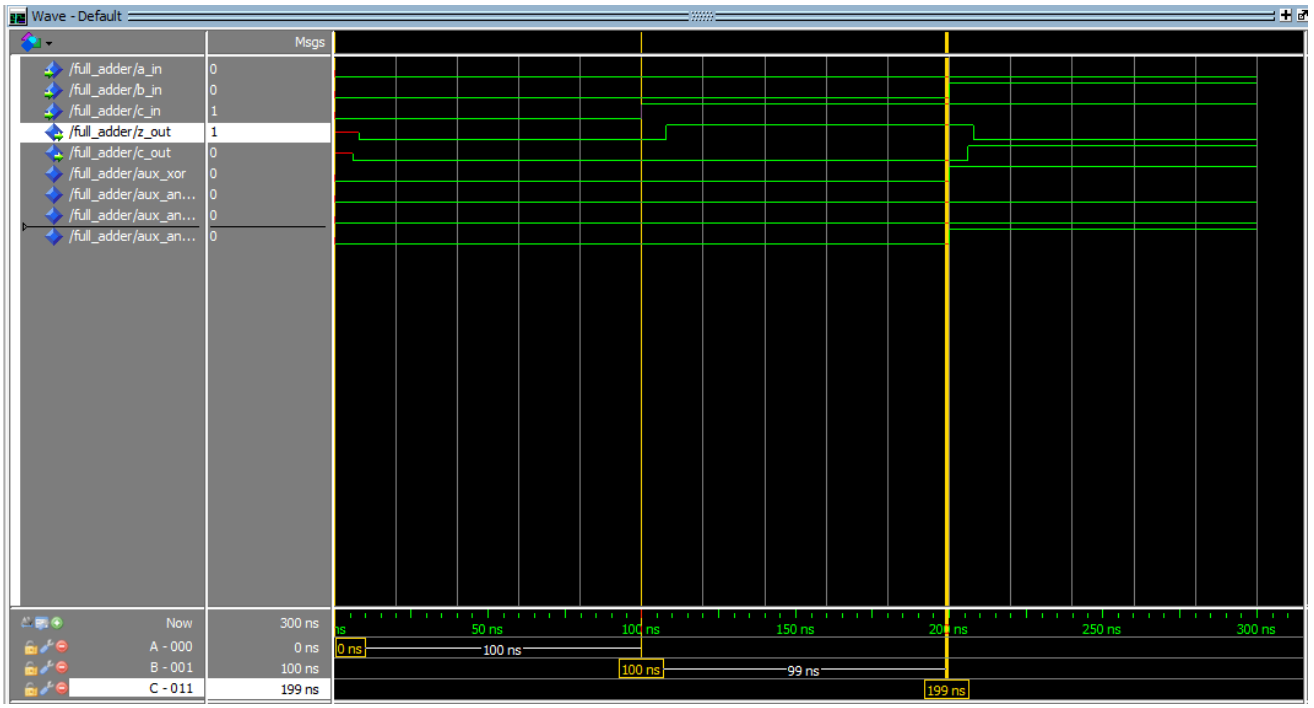

Figura 1: Hamming 1 - Todos os Estados

No momento em que a entrada c_in tem seu valor atualizado para o estado B, a saída z_out leva 8ns para atualizar seu valor:

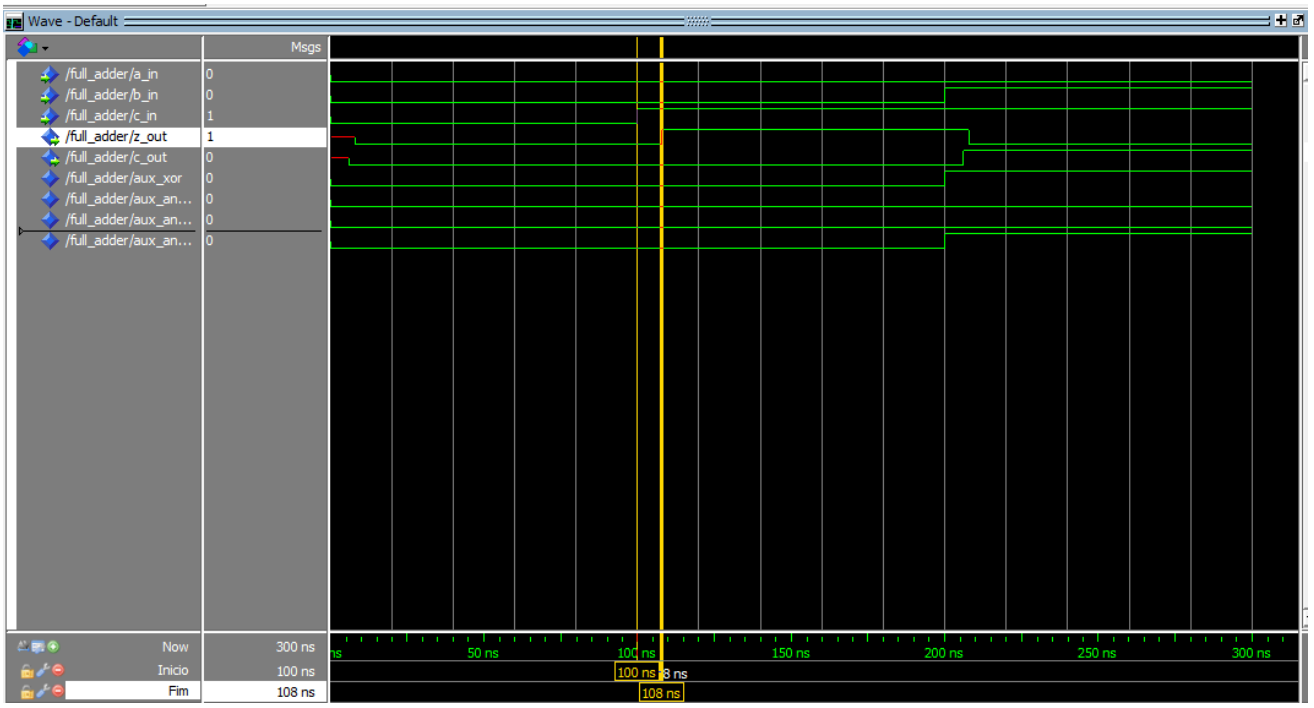

Figura 2: Hamming 1 - Estado B (001)

Já para o estado C, z_out também leva 8ns para atualizar seu valor, enquanto c_out leva 6ns:

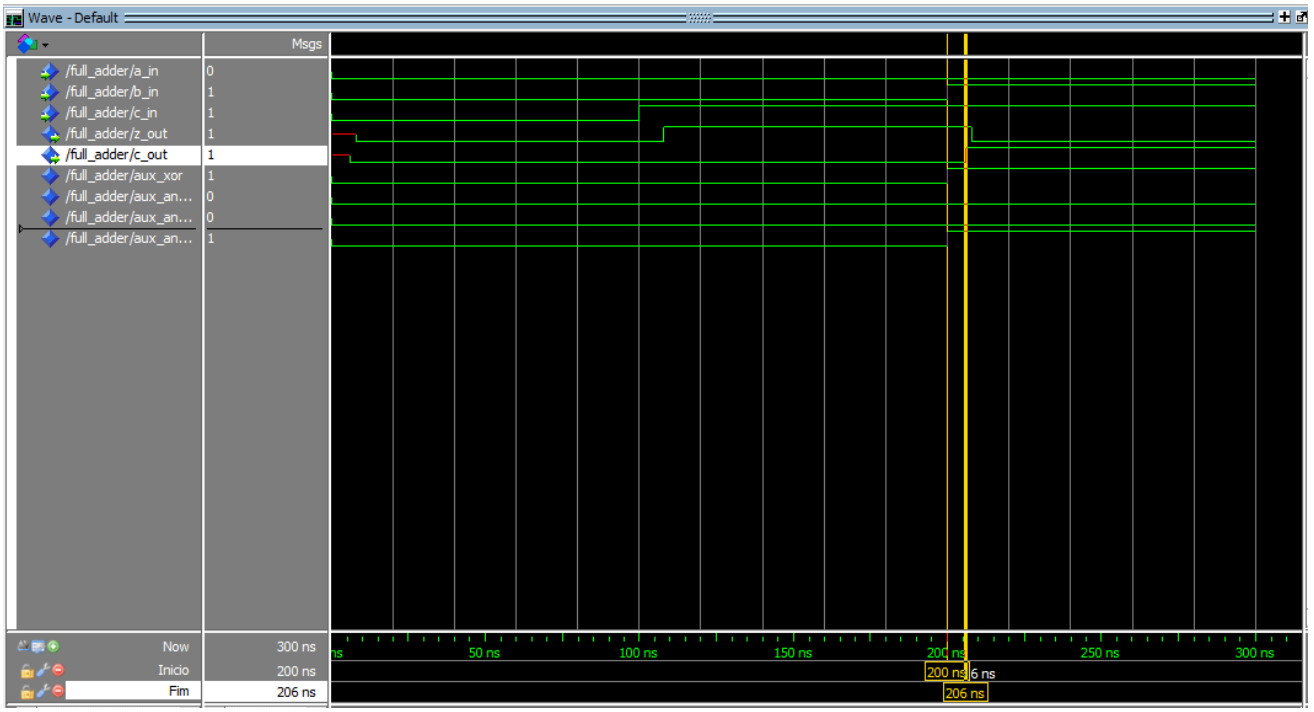

Figura 3: Hamming 1 - Estado C (011)

Como as variáveis auxiliares (aux_xor, aux_and_1, aux_and_2, aux_and_3) mudam seus valores em sincronia com as variáveis de entrada, o tempo decorrido entre suas definições e as definições de z_out e c_out são as mesmas das duas anteriores.

Em comparação com os códigos anteriores (full_adder_1 e full_adder_2 da aula anterior), os resultados são os mesmos, isso porque o propósito é o mesmo: implementar um somador completo. A diferença entre eles e o código atual é o tempo de delay entre a definição das variáveis de entrada (a_in, b_in e c_in) e definição conseguinte das variáveis intermediárias (aux_xor, aux_and_1, aux_and_2, aux_and_3, por exemplo no caso do código full_adder_3) e das variáveis de saída (z_out e c_out).

Estado	full_adder_1	full_adder_2	full_adder_3
B	4	13	8
C	8	26	8

Tabela 1: Comparativo do tempo decorrido entre a atualização da entrada (b_in ou c_in) até a atualização da saída (z_out) em ns

Essas diferenças ocorrem em decorrência do método utilizado para se implementar um somador completo. No full_adder_1, o cálculo é feito de forma concorrente com adição de *delays*. No full_adder_2 (no original e no modificado), são utilizadas entidades auxiliares, que possuem tempos de *delays* diferentes entre si, para realizar os cálculos das saídas e das variáveis intermediárias. Já no full_adder_3, é utilizado um *process* (método sequencial dentro do VHDL) com adição de delays diferentes dos códigos anteriores.

item 2) full_adder_3m_1

- **Responder: Houve erro no resultado comparado com o do full_adder_3 (selecionar uma opção)?**

Sim ☒ ☐ **Não** **(marque um 'X' na sua opção)**

- **<u>Apenas no caso de sua resposta ser Sim</u>, fazer upload de uma curva onde fique evidenciado o erro.**
- **No quadro abaixo: <u>Caso a sua resposta foi Sim</u>, explique o que está errado na curva, justificando o ocorrido. <u>Caso a sua resposta foi Não</u>, explique o porquê de as alterações no código VHDL não terem efeitos.**

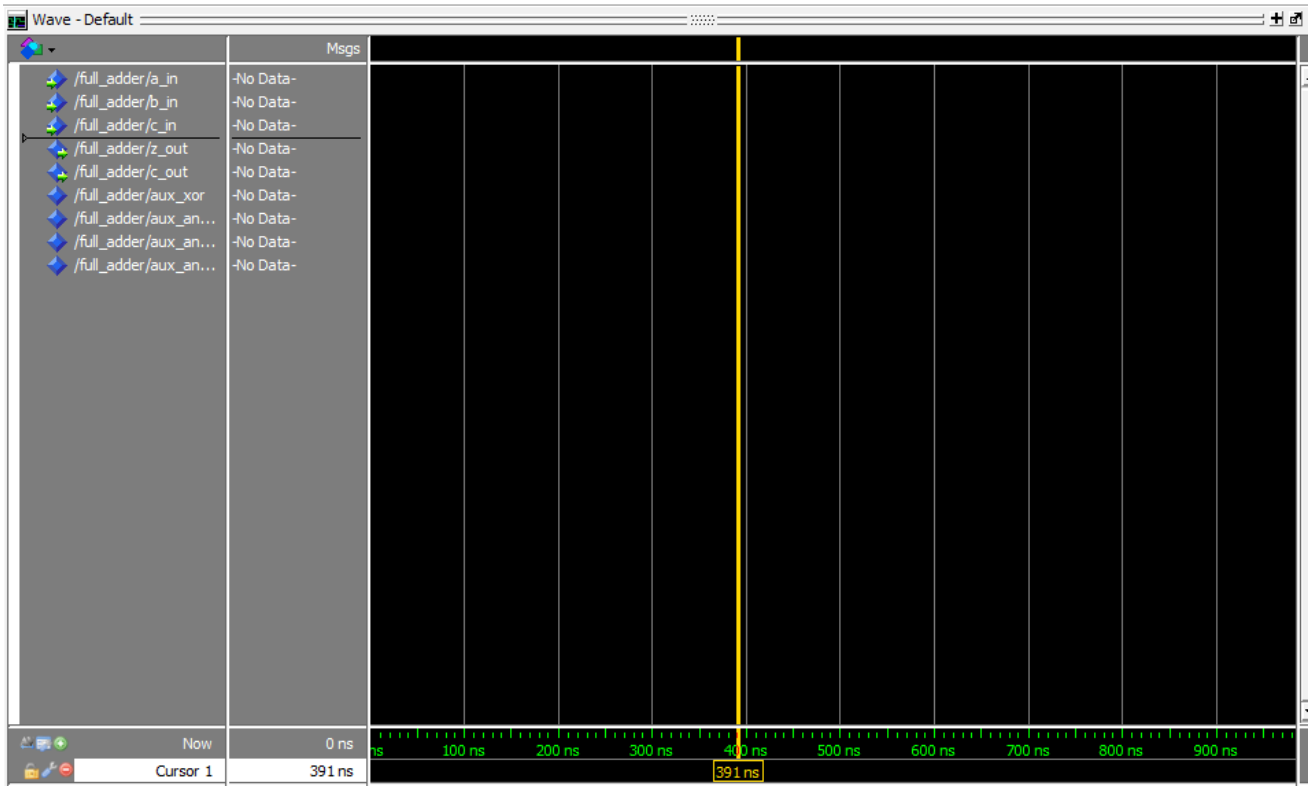

Figura 4: Simulação full_adder_3_m1.vhd

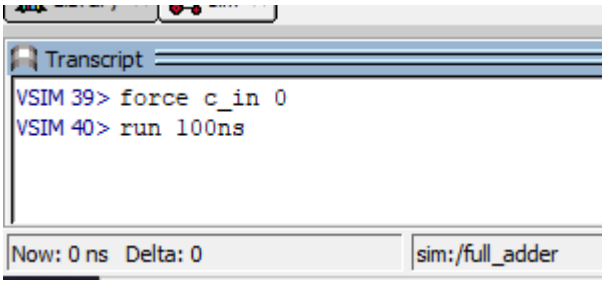

Figura 5: Transcript da Simulação full_adder_3_m1.vhd

No full_adder_3_m1, a lista de sensibilidade foi atualizada ao ter conteúdo removido. Dessa forma, o process do código inicia sua funções sequenciais, mas como não há um evento específico para funcionar como *trigger*/ativador do process, então a todo momento ele fosse executado logo após seu término, funcionando de forma semelhante a um *while()* em c/c++, nunca concluindo a atualização de fato os valores intermediários e de saídas **finais**. Isso se evidencia tanto pela falta de amostragem no gráfico Wave quanto no Transcript não indicando a finalização da simulação.

Alternativamente, é possível corrigir esse loop infinito utilizando um comando *wait until* para determinar um final do process para possibilitar que o código processe elementos além do process e que a simulação termine enfim.

item 3) full_adder_3m_2

- Responder: Houve erro no resultado comparado com o do full_adder_3 ?

Sim ☒ ☐ **Não (marque um 'X' na sua opção)**

- <u>Apenas no caso de sua resposta ser Sim</u>, fazer upload de uma curva onde fique evidenciado o erro.

- No quadro abaixo: <u>Caso a sua resposta foi Sim</u>, explique o que está errado na curva, justificando o ocorrido. <u>Caso a sua resposta foi Não</u>, explique o porquê de as alterações no código VHDL não terem efeitos.

-

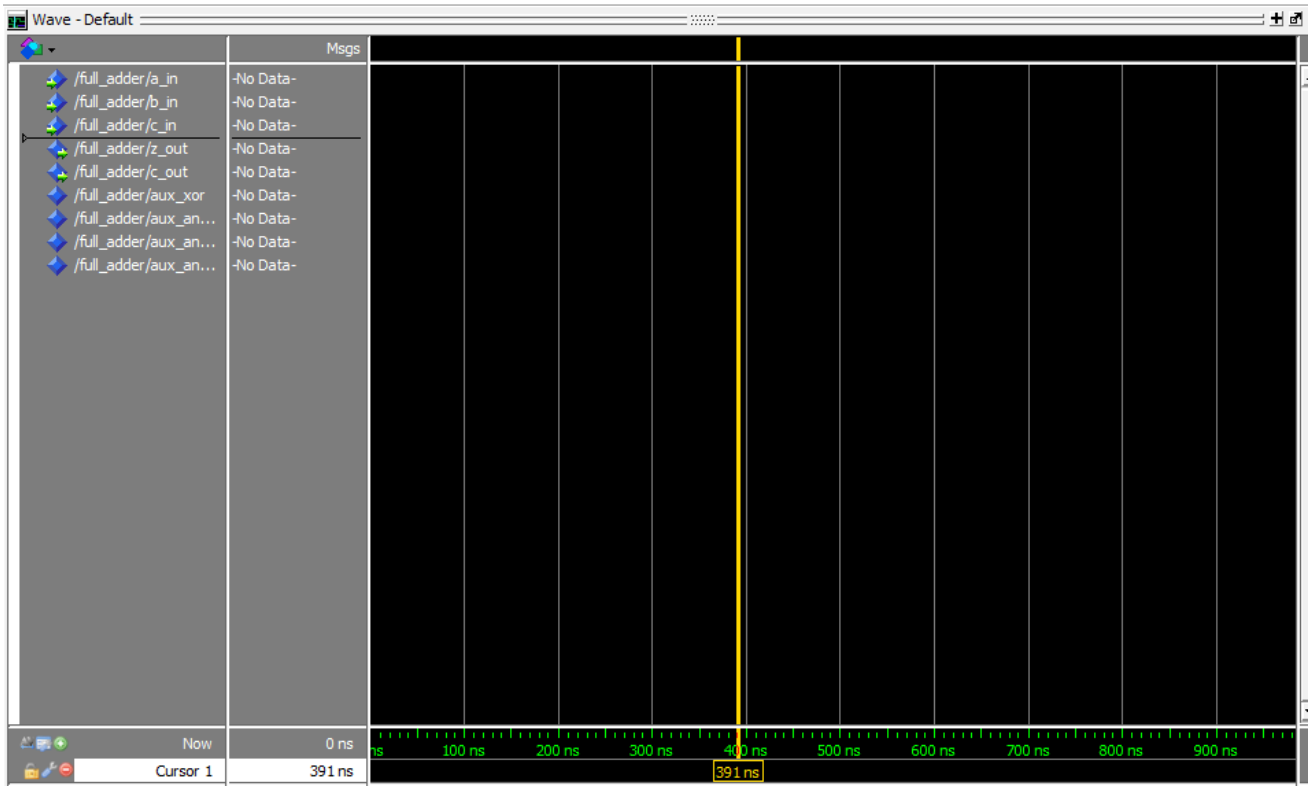

Figura 6: Simulação full_adder_3_m2.vhd

```
architecture behaviour of full_adder is

signal aux_xor, aux_and_1, aux_and_2, aux_and_3 : STD_LOGIC;

begin
    process
    begin
        c_out <= aux_and_1 or aux_and_2 or aux_and_3 AFTER 6 ns;
        z_out <= aux_xor xor c_in AFTER 8 ns;
        aux_xor <= a_in xor b_in;
        aux_and_1 <= a_in and b_in;
        aux_and_2 <= a_in and c_in;
        aux_and_3 <= b_in and c_in;
    end process;
end behaviour;
```

Figura 7: Arquitetura do full_adder_3_m2.vhd

Similar ao item anterior, o process não possui lista de sensibilidade e não utiliza um comando *wait until*. Dessa forma, a saída não pode se estabilizar num resultado final. Além disso, com as alterações realizadas neste item, as saídas z_out e c_out têm seus valores atualizados antes das variáveis intermediárias (aux_xor, aux_and_1, aux_and_2, aux_and_3), as quais elas são dependentes. Com isso, os resultados não são calculados corretamente, já que no process a lógica é sequencial e, portanto, a ordem das designações importa.

item 4) full_adder_4

- **No quadro abaixo: Compare o comportamento visto nesta simulação com o do item 1), em termos de resultado lógico e modelagem dos atrasos. Justifique.**

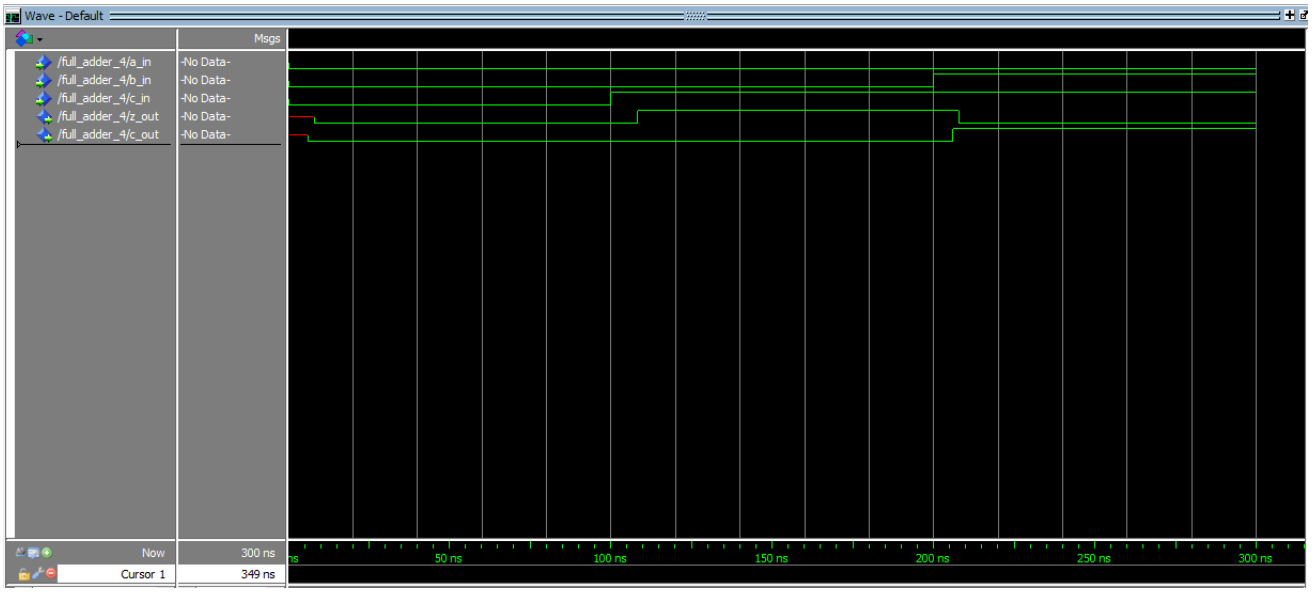

Figura 8: Simulação do full_adder_4.vhd

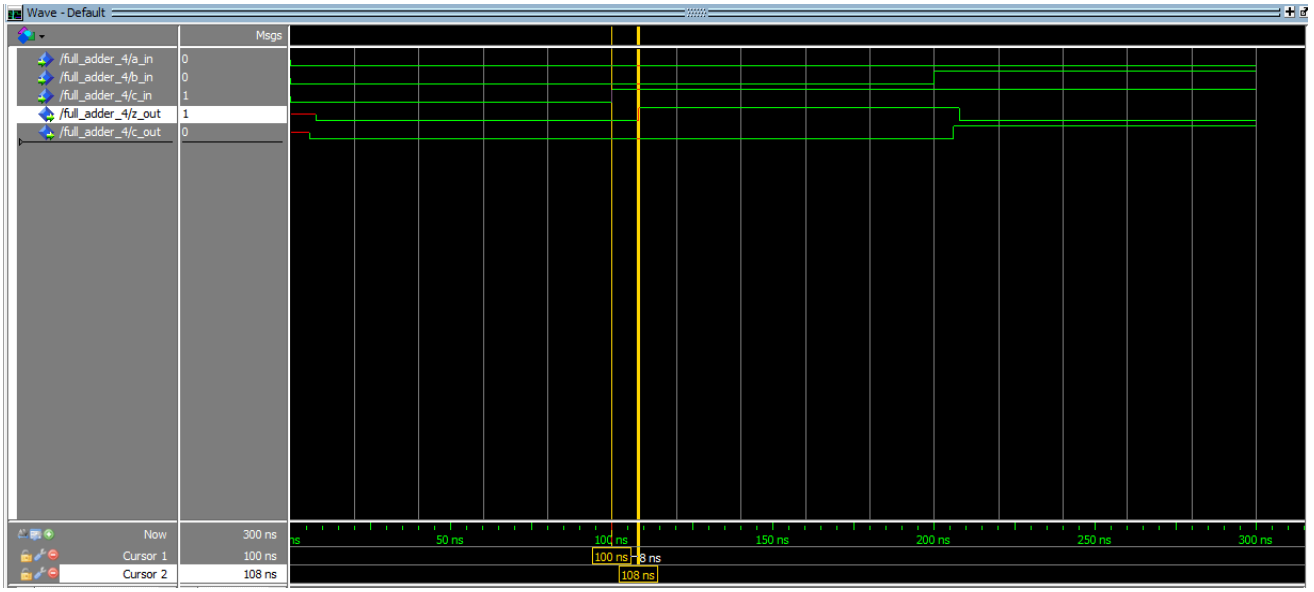

Figura 9: Estado B (Hamming 1) do full_adder_4 (delay de 8ns)

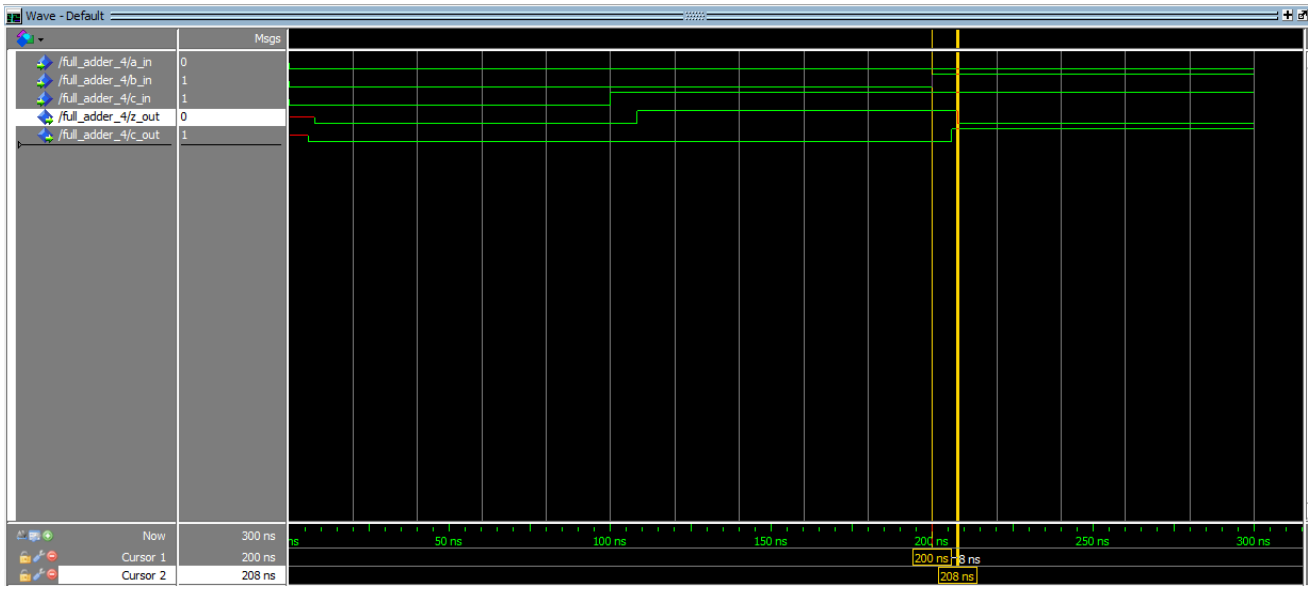

Figura 10: Estado C (Hamming 1) do full_adder_4 (delay de 8ns)

Em comparação com a simulação do full_adder_3, os resultados finais foram iguais, afinal continua sendo um circuito de somador digital completo de 1 bit. Em relação à modelagem dos atrasos, os tempos foram similares (8ns entre a definição das entradas até a definição final das saídas). Contudo, este código utiliza *variables* (escopo local e atribuição imediata) ao invés de *signals* (escopo global e atribuição a depender do código), assim, não há delays entre a definição das entradas até a definição das variáveis utilizadas no *process*, fazendo com que o delay permaneça somente atrelado às variáveis de saída (c_out e z_out) com o uso de *after x ns*.

item 5) full_adder_4_m

- Responder: Houve erro no resultado comparado com o dofull_adder_4?

Sim ☒ ☐ **Não** **(marque um 'X' na sua opção)**

- <u>Apenas no caso de sua resposta ser Sim</u>, fazer upload de uma curva onde fique evidenciado o erro.

- No quadro abaixo: <u>Caso a sua resposta foi Sim</u>, explique o que está errado na curva, justificando o ocorrido. <u>Caso a sua resposta foi Não</u>, explique o porquê de as alterações no código VHDL não terem efeitos.

Figura 11: Simulação do full_adder_4_m

A partir da definição de z_out e c_out no estado B de Hamming 1, elas se mantêm mesmo com a mudança da entrada b_in, produzindo um resultado errôneo no estado C (era esperado z_out = 0 e c_out = 1, mas foi obtido o oposto de ambos). Isso se deu porque o *process* funciona de forma sequencial e, ao mudar a ordem das quatro primeiras designações, elas tiveram seus valores atualizados somente ao final do *process* ao invés do início, alterando assim a atribuição de valores das outras variáveis e saídas.

item 6) mux- dataflow

- **Responder: A qual figura da apostila de conceito corresponde ao comportamento do módulo (selecionar uma das opções com X)?**

  ☒ **Fig.1(a)** ☐ **Fig.1(b)** ☐ **Outro**

- **Fazer upload de uma curva com o comportamento do mux, onde todas as opções de seleção apareçam.**

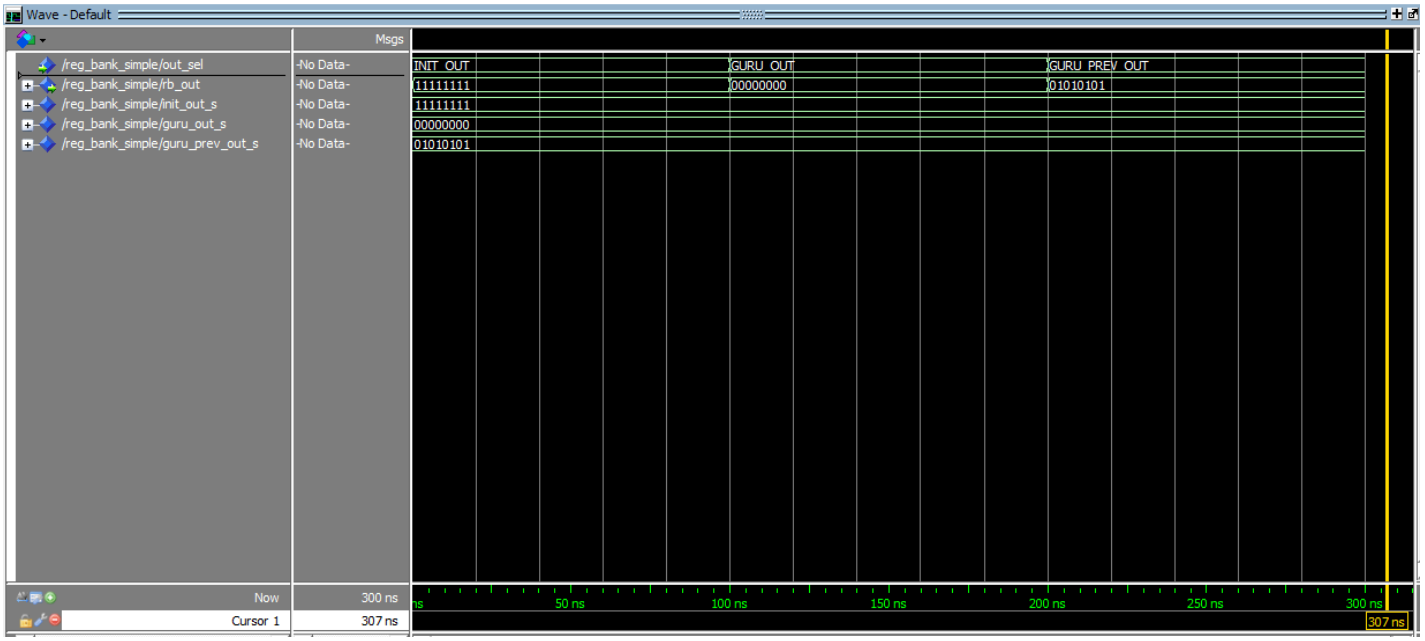

Figura 12: Simulação do reg_bank_simple_1.vhd

As saídas foram as seguintes:

out_sel	rb_out
INIT_OUT	11111111
GURU_OUT	00000000
GURU_PREV_OUT	01010101

item 7) mux- dataflow (com GURU_PREV_OUT retirado)

- **Responder: A qual figura da apostila de conceito corresponde o comportamento do módulo (selecionar uma das opções)?**

  ☐ **Fig.1(a)** ☐ **Fig.1(b)** ☒ **Outro**

- <u>Caso a opção for diferente</u> daquela do item 6):

- Fazer upload de uma curva onde fique <u>evidenciado</u> o caso diferente.

- Explique no quadro abaixo no que consiste a diferença, justificando o ocorrido: :

Figura 13: Simulação do reg_bank_simple_1_m.vhd

Com a eliminação da linha correspondente à seleção "GURU_PREV_OUT", a saída rb_out não assumiu o valor "01010101" (fig.1a) ou nem manteve o valor anterior (fig.1b). Dessa forma, como a estrutura do *when-else* prevê a existência de um *catch-all* (when others => X), ao introduzir uma entrada diferente das previstas, ele assume um valor X, como ocorreu.

item 8) mux- processes CASE

- Responder: A qual figura da apostila de conceito corresponde o comportamento do módulo (selecionar uma das opções)?

☒ Fig.1(a) ☐ **Fig.1(b)** ☐ **Outro**

- <u>Caso a opção for diferente</u> daquela dos itens 6) ou 7):

- Fazer upload de uma curva onde fique <u>evidenciado</u> o caso diferente.

- Explique no quadro abaixo no que consiste a diferença, justificando o ocorrid

9) mux- process CASE(com GURU_PREV_OUT retirado)

- **Responder: A qual figura da apostila de conceito corresponde o comportamento do módulo (selecionar uma das opções)?**

☐ **Fig.1(a)** ☒ **Fig.1(b)** ☐ **Outro**

- **<u>Caso a opção for diferente</u> daquela dos itens 6) ou 7) ou 8):**

  - **Fazer upload de uma curva onde fique <u>evidenciado</u> o caso diferente.**

  - **Explique no quadro abaixo no que consiste a diferença, justificando o ocorrido:**

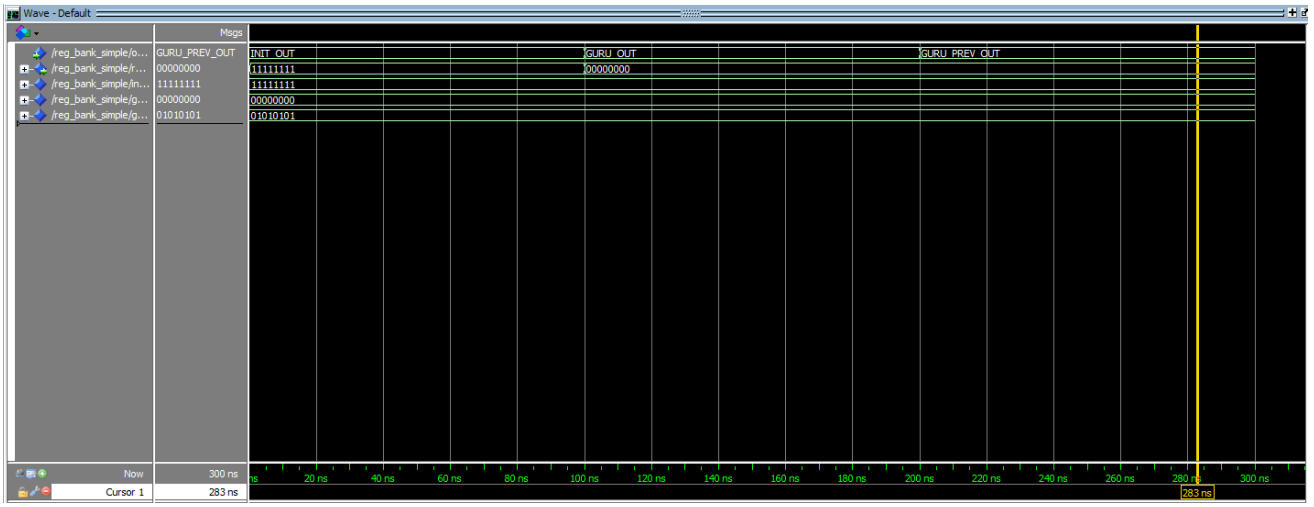

Figura 14: Simulação do reg_bank_simple_2_m.vhd

Ao usar a estrutura *case-when*, a lógica de seleção da saída é invertida: ao invés de se definir a saída a depender da entrada, a entrada define a saída a depender de seu valor. Neste caso, ao se inserir uma entrada "GURU_PREV_OUT", a saída se anula, repetindo o resultado para a entrada anterior "GURU_OUT".

item 10) Simulações com step_counter

a) **Fazer upload de uma curva que <u>mostre a ação do reset</u>. Anote e indique na própria curva como fica evidenciado que trata-se de um reset síncrono ou assíncrono.**

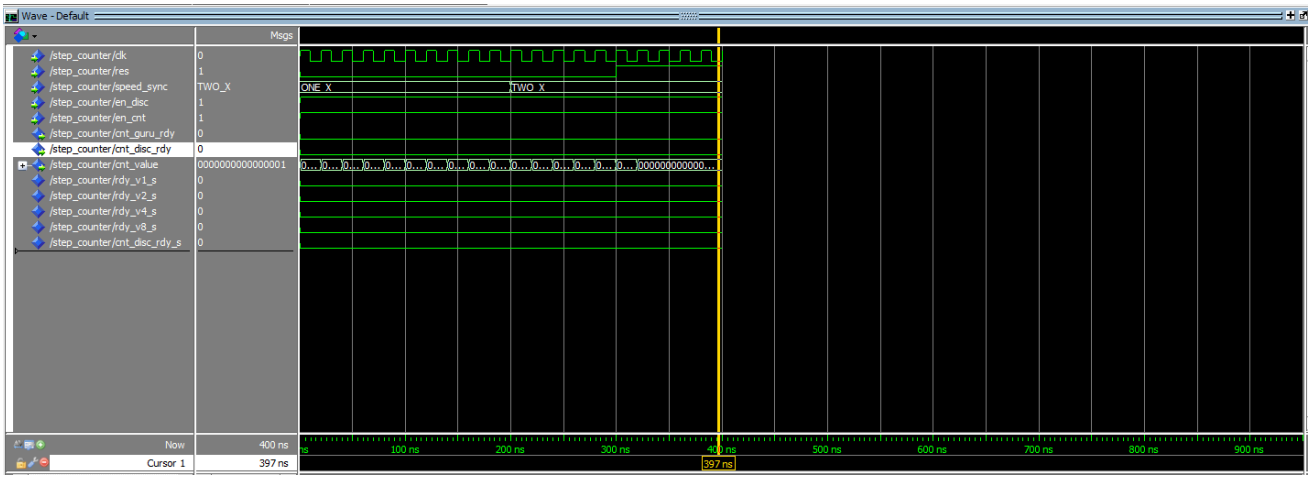

Figura 15: Simulação do step_counter_1.vhd

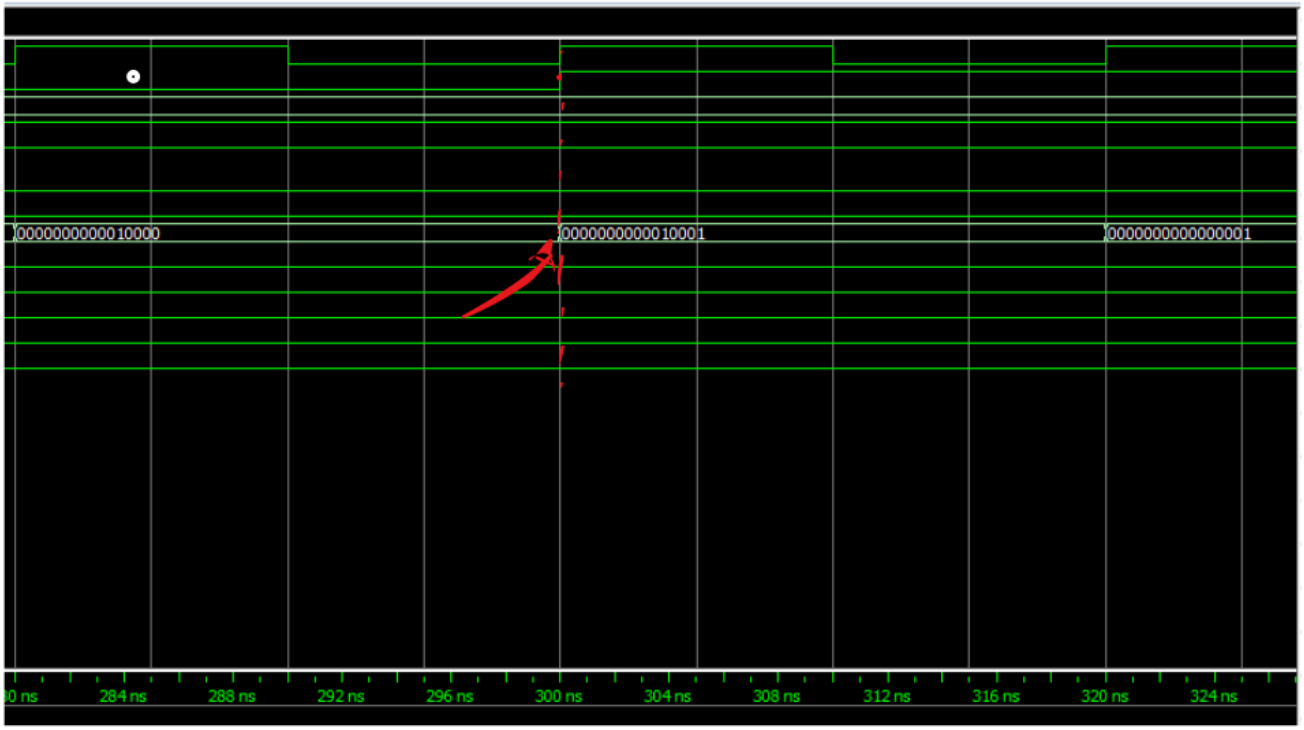

Figura 16: Sincronia do Reset com o Clock

De acordo com as duas figuras anteriores, evidencia-se que o reset utilizado é síncrono.

Para os itens b) e c), use os valores multiplicativos **VA= X4 ; VB= X8**:

b) **Fazer upload de uma curva com o caso de contagem onde o discípulo tenha velocidade VA. Anote e indique na própria curva os disparos de *cnt_guru_rdy* e *cnt_disc_rdy* que evidenciem a diferença das velocidades do guru e do discípulo.**

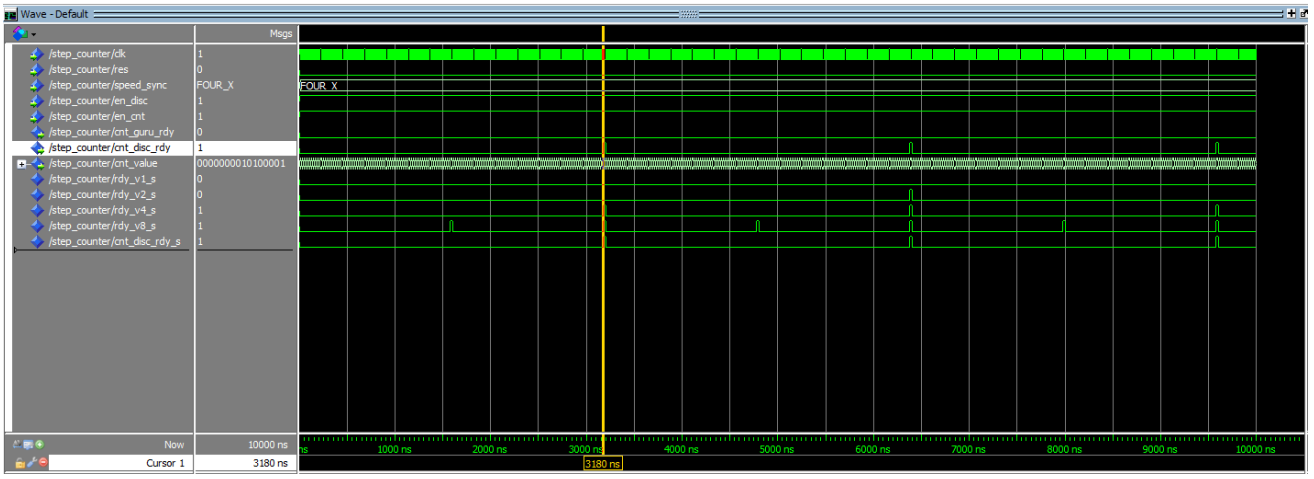

Figura 17: Simulação step_counter_1.vhd com speed = x4

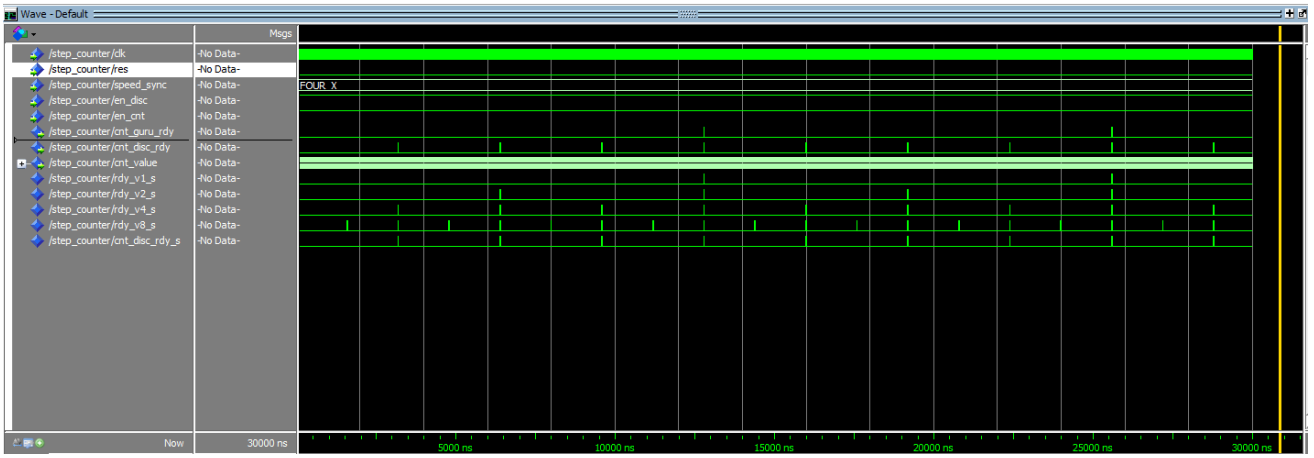

Figura 18: Simulação step_counter_1.vhd com speed = x4 (ênfase nos dois contadores)

Enquanto o contador do discípulo conta pela primeira vez depois de 3180ns do início da simulação (159 ciclos de clock a 20ns de período), o guru vai contar pela primeira vez somente depois de 12780ns depois do início da simulação (639 ciclos de clock). Dessa forma $\frac{639}{159} \approx 4\ contagens\ discípulo/contagem\ guru.$

c) **Fazer upload de uma curva com o caso de contagem onde o discípulo tenha velocidade VB,. Anote e indique na própria curva os disparos de *cnt_guru_rdy* e *cnt_disc_rdy* que evidenciem a diferença das velocidades do guru e do discípulo.**

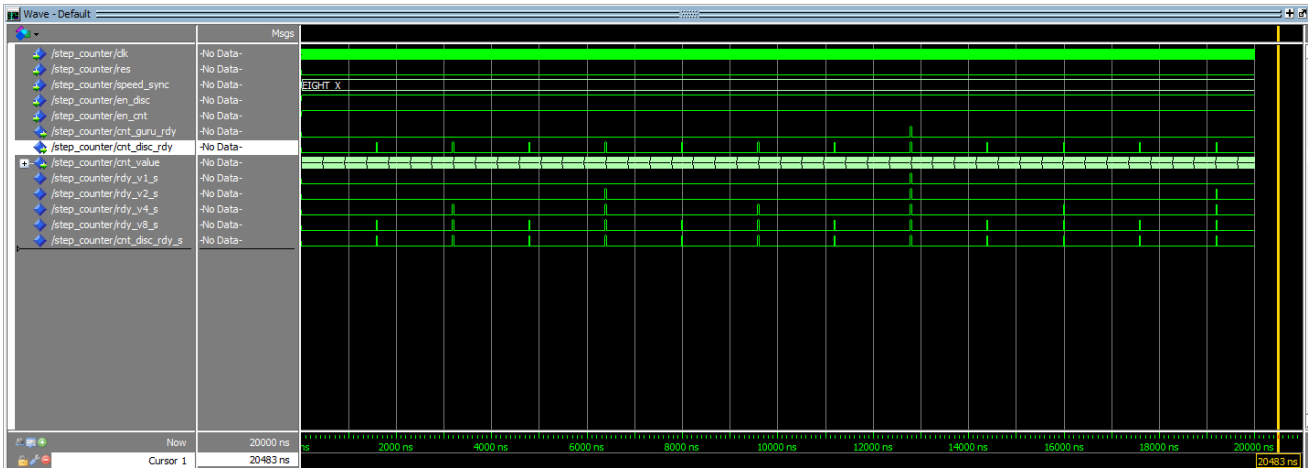

Figura 19: Simulação step_counter_1.vhd com speed = x8 (ênfase nos dois contadores)

Neste caso, o discípulo conta a cada 1580ns (79 ciclos de clock), enquanto o guru conta a cada 12780ns (639 ciclos de clock). Dessa forma, $\frac{639}{79} \approx 8\ contagens\ discípulo/contagem\ guru.$